

Algorithm Cheat Sheet · Which to Use

Find the first matching clue, then take the technique it points to. *Italic phrases under each pill are the words to look for in the problem statement.* Inspired by AlgoMonster's flowchart.

Graphs & trees

Tree structure?

Level order or min depth?
Yes **BFS**
shortest unweighted path · level order

Count or build tree shapes?
Yes **Divide & Conquer / Tree DP**
count distinct trees · subtree DP

No **DFS**
tree paths · explore all branches

Directed acyclic graph?

Yes **Topological Sort**
"course schedule" · prerequisites · DAG

Shortest path?

Weighted edges?
Yes **Dijkstra's Algorithm**
shortest path · weighted edges

No **BFS**
shortest unweighted path · level order

Connected components?

Yes **Disjoint Set Union**
connected components · "merge accounts"

Tiny input?

Yes **DFS / Backtracking**
enumerate paths · small graph

No **BFS**
shortest unweighted path · level order

Sorted, search & k-th

Sorted in, monotonic out?

Range/rank queries + updates?
Yes **Ordered Set / Fenwick / Segment Tree**
range sum + updates · order stats

No **Binary Search**
sorted array · "minimize the max"

Top-k or k-th?

Yes **Heap / Sorting**
kth largest · "top k" · running median

Linked lists

Linked list?

Fast/slow or offset pointers?
Yes **Two pointers**
pair from ends · fast/slow · in-place

Merge several sorted lists?
Yes **Heap / Divide & Conquer**
merge k sorted lists

No **Linked-List Manipulation**
reverse / reorder a list

Lookups, intervals, partition, strings

Lookup, count, or group?

Yes **Hash Table / Counting**
frequency · "first unique" · group by

Overlapping intervals?
Yes **Sorting + Interval Scan**
merge intervals · "meeting rooms"

Partition in place?

Yes **Two pointers / Partitioning**
move zeroes · Dutch flag · in-place

Match/split via dictionary?

Prefix/pattern search?
Yes **Trie / String Matching / Rolling Hash**
prefix search · "starts with"

No **Trie / DP / Memoization**
word break · dictionary split

Small constraints (n is tiny)

Brute force viable?
Yes **Brute Force / Backtracking**
all permutations / subsets · $n \leq 12$

Track a subset?
Yes **Bitmask DP**
 $n \leq 20$ · "visit all" · subset state

No **Dynamic Programming**
"number of ways" · max/min with choices

Subarrays & windows

Running sums?

Yes **Prefix Sums**
subarray sum = k · negatives allowed · cumulative totals

Contiguous span?

Window invariant?
Yes **Sliding Window**
*longest/shortest substring · $\leq k$ distinct
exactly k $\rightarrow$ atMost(k) - atMost(k-1)*

Nearest greater/smaller?

Yes **Monotonic Stack**
next greater · "daily temperatures"

No **Dynamic Programming**
"number of ways" · max/min with choices

Counting arrangements

Brute force viable?

Yes **Brute Force / Backtracking**
all permutations / subsets · $n \leq 12$

No **Dynamic Programming**
"number of ways" · max/min with choices

Several sequences?

Monotonic property?
Yes **Two pointers**
pair from ends · fast/slow · in-place

Overlapping subproblems?

Yes **Dynamic Programming**
"number of ways" · max/min with choices

Locate indices?

Monotonic property?
Yes **Two pointers**
pair from ends · fast/slow · in-place

Constant space?

Monotonic property?
Yes **Two pointers**
pair from ends · fast/slow · in-place

Compute a max / min

Monotonic search space?

Yes **Binary Search**
sorted array · "minimize the max"

Nearest greater/smaller?

Yes **Monotonic Stack**
next greater · "daily temperatures"

Overlapping subproblems?

Yes **Dynamic Programming**
"number of ways" · max/min with choices

No **Greedy Algorithms**
intervals · "jump game" · local choice

Parsing, design, simulation, math (the tail)

Parsing symbols?

Count or optimise segments?
Yes **Dynamic Programming**
"number of ways" · max/min with choices

No **Stack**
valid parentheses · expression parse

Design a structure?

Yes **Design + Supporting Structures**
"implement LRU / iterator"

Just follow the rules?

Yes **Simulation / Basic DSA**
follow the rules step by step

Math or bit tricks?

Properties of numbers?
Yes **Number Theory**
gcd / primes / modular arithmetic

No **Math / Bit Manipulation**
XOR tricks · powers · "single number"

Otherwise

Specialized / Advanced Pattern
rare / problem-specific trick

Algorithm Cheat Sheet · How They Work

The mechanism behind each technique on the front — minimal Python-ish templates, not full implementations. Badge = typical time complexity.

BFS

O(V+E)

Explore level by level. Shortest path in an unweighted graph.

```
# adj[u] = node u's neighbour nodes (v)
from collections import deque
q, seen = deque([src]), {src}
while q:
    u = q.popleft()
    for v in adj[u]:
        if v not in seen:
            seen.add(v); q.append(v)
```

DFS

O(V+E)

Go deep, then backtrack. Trees, paths, components, cycles.

```
def dfs(u):
    seen.add(u)
    for v in adj[u]:
        if v not in seen:
            dfs(v)
```

Topological Sort

O(V+E)

Order a DAG so every edge points forward (Kahn's algorithm).

```
from collections import deque
indeg = [0] * n
for u in range(n):
    for v in adj[u]: indeg[v] += 1
q = deque(u for u in range(n) if indeg[u]==0)
order = []
while q:
    u = q.popleft(); order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0: q.append(v)
# len(order) < n => graph has a cycle
```

Dijkstra

O(E log V)

Shortest path with non-negative weights, via a min-heap.

```
import heapq
dist = [float('inf')] * n; dist[src] = 0
pq = [(0, src)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]: continue
    for v, w in adj[u]:
        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))
```

Union-Find (DSU)

~O(1) amort.

Merge sets, test connectivity, detect cycles. Path compression.

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]] # compress
        x = parent[x]
    return x
def union(a, b):
    parent[find(a)] = find(b)
```

Binary Search

O(log n)

Halve a sorted (or monotonic) space. Also "search on the answer".

Example: first index $\geq x$; $[1,3,3,5]$, $x=3 \rightarrow 1$

```
def lower_bound(arr, ok): # first True index
    lo, hi = 0, len(arr) # on answer: hi=max+1
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo
```

Two Pointers

O(n)

Walk a sorted array from both ends, or fast/slow on a list.

Example: pair summing to target; $[2,7,11]$, $9 \rightarrow (0,1)$

```
def two_sum(arr, target): # arr is sorted
    i, j = 0, len(arr) - 1
    while i < j:
        s = arr[i] + arr[j]
        if s == target: return (i, j)
        if s < target: i += 1
        else: j -= 1
```

Sliding Window

O(n)

Grow right, shrink left to keep an invariant (e.g. $\leq k$ distinct).

Example: longest with $\leq k$ distinct; "eceba", $k=2 \rightarrow 3$

```
from collections import Counter
left = 0; cnt = Counter()
for right, x in enumerate(arr):
    cnt[x] += 1
    while len(cnt) > k: # shrink
        cnt[arr[left]] -= 1
        if cnt[arr[left]] == 0: del cnt[arr[left]]
        left += 1
    best = max(best, right - left + 1)
# exactly k = atMost(k) - atMost(k-1)
```

Heap / Top-k

O(n log k)

Keep the k best seen so far in a size-k min-heap.

```
import heapq
heap = [] # min-heap, size k
for x in arr:
    heapq.heappush(heap, x)
    if len(heap) > k: heapq.heappop(heap)
# heap[0] is the k-th largest
```

Merge Intervals

O(n log n)

Sort by start, then fold each interval into the last if they touch.

Example: $[[1,3],[2,6],[8,10]] \rightarrow [[1,6],[8,10]]$

```
intervals.sort()
merged = [intervals[0]]
for a, b in intervals[1:]:
    if a <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], b)
    else:
        merged.append([a, b])
```

Dynamic Programming

O(coins*amt)

Build answers from overlapping subproblems.

Example: coins $[1,2,5]$, amount 11 $\rightarrow 3$

```
# dp[t] = running best (fewest coins to make t)
dp = [0] + [float('inf')] * amount
for coin in coins: # consider one more option
    # ascending/descending => can/cannot reuse
    for t in range(coin, amount + 1):
        # update every subproblem with choice
        dp[t] = min(dp[t], dp[t-coin] + 1)
# answer = dp[amount] (inf => impossible)
```

Memoization

O(states)

Recurse on subproblems, cache each result in a dict.

Example: nth Fibonacci; fib(6) $\rightarrow 8$

```
memo = {}
def fib(num):
    if num <= 1: return num
    if num in memo: return memo[num]
    memo[num] = fib(num-1) + fib(num-2)
    return memo[num]
```

Backtracking

O(2ⁿ) / O(n!)

Choose, recurse, undo. Permutations & subsets when n is tiny.

Example: subsets of $[1,2] \rightarrow [], [1], [1,2], [2]$

```
def backtrack(start, path):
    res.append(path[:]) # record
    for i in range(start, n):
        path.append(arr[i])
        backtrack(i + 1, path) # i+1 => subsets
    path.pop() # undo
```

Monotonic Stack

O(n)

Stack holds a sorted run; pop when the new value breaks it.

Example: next greater value to the right; $[2,1,3] \rightarrow [3,3,-1]$

```
stack = []; res = [-1] * n
for i, x in enumerate(arr):
    while stack and arr[stack[-1]] < x:
        res[stack.pop()] = x
    stack.append(i)
```

Prefix Sums

O(n) build

Precompute cumulative totals for O(1) range-sum queries.

```
prefix = [0] * (n + 1)
for i, x in enumerate(arr):
    prefix[i+1] = prefix[i] + x
# sum(l..r) = prefix[r+1] - prefix[l]
# subarray=k: count prefix[] in a hashmap
```

Trie

O(len) / op

Prefix tree for word lookups and "starts-with" queries.

```
def insert(root, word):
    cur = root
    for c in word:
        cur = cur.setdefault(c, {})
    cur['$'] = True # word ends here
```